

Actividad Práctica – Sistema de Quests en Blueprints

Blueprints – Arquitectura modular de misiones

Objetivo

Desarrollar un sistema básico y reutilizable de misiones en Blueprints. El sistema deberá permitir aceptar misiones, identificar cada misión mediante un ID, manejar distintos tipos de objetivos, actualizar el progreso, completar una misión, comunicar los cambios a una interfaz y separar definición, progreso y presentación.

1. Enum de tipos de objetivo

Crear:

`EQuestObjectiveType`

- Kill
- Collect
- Interact
- ReachLocation

2. Struct de objetivo

Crear:

`ST_QuestObjective`

- ObjectiveID – Name
- ObjectiveType – EQuestObjectiveType
- TargetID – Name
- RequiredAmount – Integer
- CurrentAmount – Integer

3. Primary Data Asset de Quest

Crear una clase Blueprint basada en PrimaryDataAsset:

`PDA_QuestDefinition`

- QuestID – Name
- QuestName – Text
- Description – Text
- Objectives – Array<ST_QuestObjective>

El Data Asset deberá contener únicamente la definición de la misión. No deberá almacenar el progreso durante la partida.

4. Struct de Quest Runtime

Crear:

`ST_QuestRuntime`

- QuestID – Name
- QuestDefinition – PDA_QuestDefinition Object Reference
- Objectives – Array<ST_QuestObjective>
- IsActive – Boolean
- IsCompleted – Boolean

Este Struct almacenará el estado de la misión durante la partida.

5. Quest Component

Crear un Actor Component:

`BP_QuestComponent`

Agregarlo al Character.

Variable principal:

`ActiveQuests - Array<ST_QuestRuntime>`

6. Función FindQuestByID

Crear:

`FindQuestByID`

Input:

- QuestID – Name

Outputs:

- Found – Boolean
- QuestData – ST_QuestRuntime
- QuestIndex – Integer

La función deberá buscar una misión dentro de ActiveQuests utilizando su QuestID.

7. Función AddQuest

Crear:

`AddQuest`

Input:

- QuestDefinition – PDA_QuestDefinition Object Reference

La función deberá:

- obtener el QuestID;
- comprobar si la misión ya existe;
- crear un ST_QuestRuntime;
- copiar los objetivos;
- establecer IsActive;
- establecer IsCompleted;
- agregar la misión a ActiveQuests.

8. Blueprint Interface

Crear:

BPI_Interactable

Función:

Interact

Input:

- Interactor – Actor

9. Quest Giver

Crear:

BP_QuestGiver

Variable:

- QuestDefinition – PDA_QuestDefinition Object Reference

El Actor deberá implementar BPI_Interactable. Cuando recibe Interact, deberá entregar la misión al BP_QuestComponent del jugador.

10. GameState

Crear:

BP_QuestGameState

Basado en GameStateBase.

Crear un Event Dispatcher:

OnGameplayObjectiveEvent

Parámetros:

- ObjectiveType – EQuestObjectiveType
- TargetID – Name
- Amount – Integer

11. Función ReportGameplayObjective

Dentro de BP_QuestGameState crear:

ReportGameplayObjective

Inputs:

- ObjectiveType – EQuestObjectiveType
- TargetID – Name
- Amount – Integer

La función deberá llamar al Event Dispatcher OnGameplayObjectiveEvent.

12. Eventos de gameplay

Los Actors del mundo que puedan generar progreso deberán reportarlo mediante ReportGameplayObjective.

Podrán existir Actors asociados a:

- Kill
- Collect
- Interact
- ReachLocation

Cada Actor deberá tener un identificador propio de tipo Name.

13. Suscripción del Quest Component

En BP_QuestComponent, durante BeginPlay:

- obtener el GameState;
- convertirlo a BP_QuestGameState;
- hacer Bind a OnGameplayObjectiveEvent.

Crear un Custom Event:

HandleGameplayObjective

Inputs:

- ObjectiveType – EQuestObjectiveType
- TargetID – Name
- Amount – Integer

14. HandleGameplayObjective

HandleGameplayObjective deberá:

- recorrer ActiveQuests;
- procesar únicamente las quests activas;
- ignorar las quests completadas;
- recorrer los Objectives;

- comparar ObjectiveType;
- comparar TargetID;
- llamar a UpdateObjectiveProgress cuando corresponda.

15. Función UpdateObjectiveProgress

Crear:

UpdateObjectiveProgress

Inputs:

- QuestIndex – Integer
- ObjectiveIndex – Integer
- Amount – Integer

La función deberá:

- obtener la Quest desde ActiveQuests;
- obtener una copia de Objectives;
- obtener el objetivo mediante ObjectiveIndex;
- calcular el nuevo CurrentAmount;
- impedir que supere RequiredAmount;
- actualizar el ST_QuestObjective;
- reemplazarlo dentro de Objectives;
- actualizar el ST_QuestRuntime;
- reemplazar la Quest dentro de ActiveQuests.

16. Función CheckQuestCompleted

Crear:

CheckQuestCompleted

Input:

- QuestIndex – Integer

La función deberá comprobar todos los objetivos de la Quest.

Cada objetivo estará completo cuando:

`CurrentAmount >= RequiredAmount`

Si todos están completos:

- IsCompleted = True
- IsActive = False

La Quest actualizada deberá volver a guardarse dentro de ActiveQuests.

17. Event Dispatchers del Quest Component

Crear dentro de BP_QuestComponent:

OnQuestStarted

- QuestData – ST_QuestRuntime

OnQuestProgressChanged

- QuestData – ST_QuestRuntime

OnQuestCompleted

- QuestData – ST_QuestRuntime

18. Interfaz de Quest

Crear:

WBP_QuestTracker

La interfaz deberá mostrar:

- nombre de la Quest;
- descripción;
- objetivos;
- progreso actual;
- cantidad requerida;
- estado de la misión.

La UI deberá reaccionar a los Event Dispatchers del BP_QuestComponent. No deberá consultar constantemente los datos mediante Tick.

19. Requisitos mínimos

El sistema deberá utilizar:

- Enum.
- Struct.
- Array de Structs.
- Primary Data Asset.
- Actor Component.
- Blueprint Interface.
- GameState.
- Event Dispatcher.
- For Each Loop.
- Set Members in Struct.
- Set Array Elem.
- Widget.
- IDs utilizando Name.

20. Restricciones

- No utilizar Tick para actualizar quests o UI.
- No modificar el PDA_QuestDefinition durante la partida.
- Los Actors del mundo no deberán modificar directamente ActiveQuests.
- La UI no deberá modificar el progreso.
- El QuestGiver no deberá controlar el progreso después de entregar la misión.
- La lógica principal deberá permanecer dentro de BP_QuestComponent.
- El GameState deberá utilizarse únicamente para reportar eventos globales relacionados con objetivos.

21. Flujo esperado

QuestGiver → AddQuest → ActiveQuests

Actor del mundo → ReportGameplayObjective → BP_QuestGameState →
OnGameplayObjectiveEvent

OnGameplayObjectiveEvent → HandleGameplayObjective → UpdateObjectiveProgress →
CheckQuestCompleted

CheckQuestCompleted → OnQuestProgressChanged / OnQuestCompleted → WBP_QuestTracker